МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 3 по курсу
«Операционные системы и архитектура компьютера»

Выполнила: Дарья Новикова Витальевна
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 08.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: освоить создание и синхронизацию потоков POSIX, реализовать многопоточную свёртку вещественной матрицы и исследовать ускорение и эффективность при разном размере данных и числе потоков.

Задание: написать программу на C, которая K раз применяет фильтр свёртки к матрице вещественных чисел. Размер окна задаёт пользователь, максимальное число одновременно работающих потоков задаётся ключом запуска.

Вариант 13: наложить K раз фильтр, использующий матрицу свёртки, на матрицу вещественных чисел; размер окна задаётся пользователем.

Метод решения

Исходная матрица считывается из input.txt. Выбран усредняющий фильтр: каждый коэффициент нечётного окна $w \times w$ равен $1/w^2$; значения за границами матрицы считаются нулевыми. В каждом проходе p рабочих потоков делят строки выходной матрицы на непересекающиеся диапазоны. Они только читают общую входную матрицу и записывают свои строки результата. Главный поток вызывает pthread_join для каждого рабочего, записывает готовую матрицу в output.txt и меняет роли входного и выходного буферов. Для $p>1$ программа отдельно измеряет тот же расчёт на копии исходных данных одним рабочим потоком; $S=T_1/T_p$, $E=S/p$.

Описание программы

Проект состоит из небольших модулей, конфигурации сборки и текстовых файлов данных. main управляет вводом параметров и K проходами; matrix_io читает исходную матрицу; benchmark измеряет время и однопоточный запуск; filter вычисляет свёртку и управляет рабочими потоками.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
main.c	Параметры запуска, K проходов и вывод результатов.
matrix_io.c / .h	Чтение исходной матрицы из input.txt.
benchmark.c / .h	Замер времени и контрольный запуск с одним потоком.
filter.c	Расчёт свёртки, создание и ожидание потоков POSIX.
filter.h	Структура Worker и объявление filter_pass.
CMakeLists.txt	Автоматизированная сборка программы через CMake.
input.txt	Исходная матрица; первая строка задаёт размеры.
output.txt	История проходов, времена, ускорение и эффективность.

Основные типы данных и функции

Worker хранит адреса входной и выходной матриц, размеры, окно и диапазон строк [first_row, last_row). read_matrix из matrix_io возвращает размер и данные файла; worker_main вычисляет свои строки; filter_pass создаёт потоки и ожидает их; run_timed_pass из benchmark измеряет один проход; measure_one_worker измеряет все K проходов на копии данных одним потоком.

Используемые системные вызовы

pthread_create и pthread_join являются функциями библиотеки POSIX, а не отдельными системными вызовами. В трассировке Linux создание потоков проявляется через clone3, ожидание — через futex. Работа с файлами и TID также видна на уровне ядра.

Таблица 2 – API и системные вызовы

API / вызов	Назначение
pthread_create / clone3	Создание рабочего потока; clone3 виден в strace.
pthread_join / futex	Ожидание завершения рабочего потока.
gettid	Получение TID рабочего потока.
openat, read, close	Открытие, чтение и закрытие input.txt.
openat, write, close	Открытие, запись и закрытие output.txt.
clock_gettime	Измерение длительности проходов; может выполняться через vDSO.

Ключевые фрагменты кода

Разделение строк исключает одновременную запись потоков в одну клетку. Функция filter_pass создаёт каждый поток, проверяет код возврата и ожидает все успешно созданные потоки, даже если последующее создание завершилось ошибкой.

Листинг 1 – Создание и ожидание потоков (filter.c)

```
for (; created < count; ++created) {
    int error = pthread_create(&threads[created], NULL,
                               worker_main, &workers[created]);
    if (error != 0) { failed = 1; break; }
}
for (size_t i = 0; i < created; ++i) {
    int error = pthread_join(threads[i], NULL);
    if (error != 0) failed = 1;
}
/* Один поток записывает только свой диапазон строк. */
work->output[row * work->cols + col] = sum;
return failed ? -1 : 0;
```

Результаты

Корректность проверена на матрице 3×3 из единиц при окне 3×3 и K=2. Первый проход даёт в центре 1, на углах 4/9; после второго прохода центральный элемент равен 49/81 ≈ 0,604938. Результаты расчёта для 1 и 2 потоков совпали. Пример запуска и отдельный снимок потоков ОС приведены ниже.

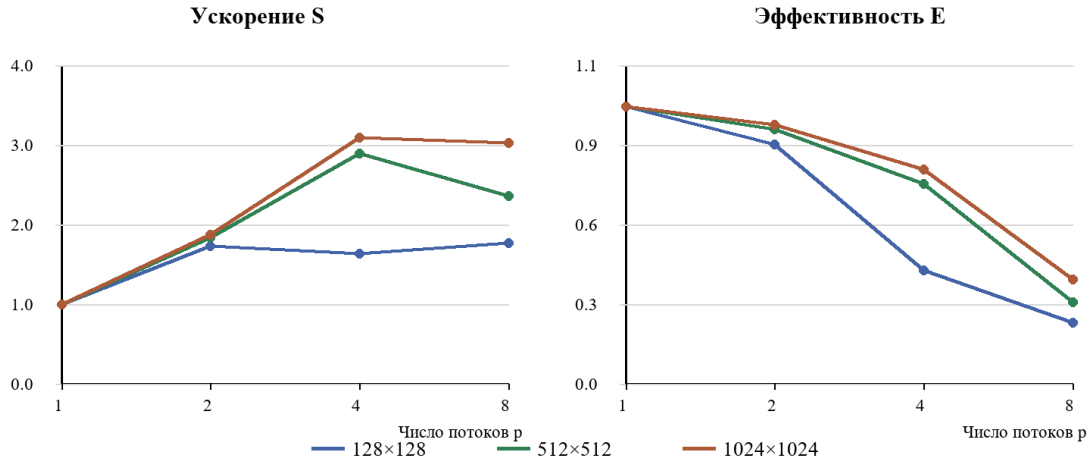
```
$ ./build/lab3 -t 2
Enter window size: 3; Enter number of filter applications: 2
Pass 2 center: 0.604938; S = T1/T2; E = S/2
Worker 1 and Worker 2: one PID, different TID
$ ps -L -p 42450 -o pid,tid,stat,comm
PID     TID STAT COMMAND
  42450   42450 Sl+  lab3
  42450   42470 Rl+  lab3
  42450   42471 Rl+  lab3
  42450   42472 Rl+  lab3
  42450   42473 Rl+  lab3
```

Для каждой конфигурации выполнено пять запусков. В таблице приведена медиана времени T_p; S и E рассчитаны из медиан T_1 и T_p для одного размера. Матрицы заполнялись по правилу a[i]=(i mod 97)/10; окно 7×7, K=3. Времена включают создание и ожидание потоков, но исключают чтение и запись матриц. Среда: Ubuntu 24.04 в WSL2, 4 логических процессора, GCC 13.3.0, сборка Release.

Таблица 3 – Время, ускорение и эффективность при разных размерах

Матрица / p	Время T_p, мс	Ускорение S	Эффективность E
128×128 / 1	1.714	1.000	1.000
128×128 / 2	0.991	1.729	0.864
128×128 / 4	1.049	1.634	0.409
128×128 / 8	0.972	1.763	0.220
512×512 / 1	22.293	1.000	1.000
512×512 / 2	12.148	1.835	0.918
512×512 / 4	7.714	2.890	0.722
512×512 / 8	9.464	2.356	0.294
1024×1024 / 1	88.213	1.000	1.000
1024×1024 / 2	47.162	1.870	0.935
1024×1024 / 4	28.549	3.090	0.772
1024×1024 / 8	29.212	3.020	0.377

Рисунок 1 – Ускорение и эффективность в зависимости от числа потоков



Для матрицы 1024×1024 четыре потока дают наибольшее медианное ускорение 3,090. При восьми потоках на четырёх логических процессорах дополнительного ускорения не получено; эффективность снижается.

Демонстрация работы strace

Трассировка выполнена для матрицы 3×3, окна 3×3, K=1 и -t 2. В команду включены вызовы создания и ожидания потоков, получения TID и файлового ввода-вывода. Ниже приведены ключевые строки; полный вывод именно этой команды находится в приложении Б.

```
$ strace -f -o strace.txt -e trace=clone,clone3,futex,gettid,openat,close,read,write ./build/lab3 -t 2
clone3(... CLONE_THREAD ... parent_tid=[42391]) = 42391
clone3(... CLONE_THREAD ... parent_tid=[42392]) = 42392
```

clone3 создаёт два рабочих потока фактического прохода. До них в трассировке виден ещё один поток для контрольного расчёта T_1; это отдельный последовательный этап, а не превышение лимита одновременной работы.

```
42391 gettid() = 42391
42385 futex(... FUTEX_WAIT_BITSET ..., 42392, ...) = 0
```

gettid получает TID рабочего. Ожидание futex соответствует блокировке главного потока при pthread_join. Вызовы openat/read/close относятся к input.txt, openat/write/close — к output.txt.

Выводы

Реализована K-кратная свёртка вещественной матрицы с POSIX-потоками и ограничением их числа ключом -t. Совпадение результатов при 1 и 2 потоках подтвердило корректность разделения строк и синхронизации проходов. Для матрицы 1024×1024 медианное ускорение при 4 потоках составило 3,090, эффективность — 0,772; при 8 потоках эффективность снизилась до 0,377 на системе с 4 логическими процессорами. На малой матрице времена сильнее колеблются, поскольку затраты на создание потоков становятся заметными относительно самого расчёта.

Приложение А. Исходный код программы

Полные тексты исходных и сборочных файлов проекта приведены ниже. Значимые для работы фрагменты отдельно пояснены в основном тексте.

Листинг А.1 – Файл main.c

```
#include "filter.h"
#include "benchmark.h"
#include "matrix_io.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
int main(int argc, char **argv)
{
    size_t limit;
    if (argc != 3 || strcmp(argv[1], "-t") != 0 || argv[2][0] == '-' ||
        sscanf(argv[2], "%zu", &limit) != 1 || limit == 0) {
        fprintf(stderr, "Invalid input\n");
        return EXIT_FAILURE;
    }
    size_t rows = 0;
    size_t cols = 0;
    double *input = NULL;
    if (read_matrix(&rows, &cols, &input) != 0)
        return EXIT_FAILURE;
    size_t window, iterations;
    printf("Enter window size:\n");
    if (scanf("%zu", &window) != 1 || window <= 0 || window > rows ||
        window > cols || window % 2 == 0) {
        fprintf(stderr, "Window must be odd and not larger than the matrix\n");
        free(input);
        return EXIT_FAILURE;
    }
    printf("Enter number of filter applications:\n");
    if (scanf("%zu", &iterations) != 1 || iterations <= 0) {
        fprintf(stderr, "K must be a positive number\n");
        free(input);
        return EXIT_FAILURE;
    }
    size_t count = rows * cols;
    // because we can have more workers than rows, we limit the number of workers to the number of r
ows
    size_t worker_count = limit < rows ? limit : rows;
    double *output = malloc(count * sizeof(*output));
    Worker *workers = malloc(worker_count * sizeof(*workers));
    if (output == NULL || workers == NULL) {
        perror("malloc");
        free(input); free(output); free(workers);
        return EXIT_FAILURE;
    }
    double one_worker_seconds = 0.0;
    if (worker_count > 1 && measure_one_worker(input, output, rows, cols, window, iterations, &one_w
orker_seconds) != 0) {
        free(input); free(output); free(workers);
        return EXIT_FAILURE;
    }
    FILE *output_file = fopen("output.txt", "a");
    if (output_file == NULL) {
        perror("output.txt");
        free(input); free(output); free(workers);
        return EXIT_FAILURE;
    }
    fprintf(output_file, "\nMatrix %zu x %zu, window = %zu, K = %zu, workers = %zu\n", rows, cols, w
indow, iterations, worker_count);
    double total_seconds = 0.0;
    int failed = 0;
    for (size_t pass = 1; pass <= iterations; ++pass) {
        for (size_t i = 0; i < worker_count; ++i) {
            workers[i] = (Worker){
                .input = input, .output = output, .rows = rows, .cols = cols, .window = window,
                .first_row = i * rows / worker_count, .last_row = (i + 1) * rows / worker_count
            };
        }
```

```
        if (run_timed_pass(workers, worker_count, &total_seconds) != 0) {
            failed = 1;
            break;
        }
        fprintf(output_file, "Pass %zu:\n", pass);
        for (size_t row = 0; row < rows; ++row) {
            for (size_t col = 0; col < cols; ++col)
                fprintf(output_file, "%.6f%c", output[row * cols + col], col + 1 == cols ? '\n' : '
');
        }
        if (ferror(output_file)) {
            perror("writing output.txt");
            failed = 1;
            break;
        }
        double *temporary = input;
        input = output;
        output = temporary;
    }
    if (!failed) {
        if (worker_count == 1)
            one_worker_seconds = total_seconds;
        if (total_seconds <= 0.0) {
            fprintf(stderr, "Measured time is too short\n");
            failed = 1;
        } else {
            double speedup = one_worker_seconds / total_seconds;
            double efficiency = speedup / (double)worker_count;
            fprintf(output_file, "T1: %.9f s; T%zu: %.9f s\nSpeedup: %.6f; Efficiency: %.6f\n", one_
worker_seconds, worker_count, total_seconds,                  speedup, efficiency);
            for (size_t i = 0; i < worker_count; ++i)
                printf("Worker %zu: PID=%ld TID=%ld\n",
                       i + 1, (long)getpid(), (long)workers[i].tid);
        }
        if (ferror(output_file)) {
            perror("writing output.txt");
            failed = 1;
        }
    }
    if (fclose(output_file) != 0) {
        perror("closing output.txt");
        failed = 1;
    }
    free(input);
    free(output);
    free(workers);
    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Листинг А.2 – Файл matrix_io.c

```
#include "matrix_io.h"
#include <stdio.h>
#include <stdlib.h>
int read_matrix(size_t *rows, size_t *cols, double **matrix)
{
    FILE *file = fopen("input.txt", "r");
    if (file == NULL) {
        perror("input.txt");
        return -1;
    }
    int failed = 0;
    if (fscanf(file, "%zu %zu", rows, cols) != 2 || *rows == 0 || *cols == 0) {
        fprintf(stderr, "Invalid matrix size in input.txt\n");
        failed = 1;
    }
    if (!failed) {
        *matrix = malloc(*rows * *cols * sizeof(**matrix));
        if (*matrix == NULL) {
            perror("malloc matrix");
            failed = 1;
        }
    }
    for (size_t i = 0; !failed && i < *rows * *cols; ++i) {
        if (fscanf(file, "%lf", &(*matrix)[i]) != 1) {
            fprintf(stderr, "Invalid matrix value at position %zu\n", i);
            failed = 1;
```

```
        }
    }
    if (fclose(file) != 0) {
        perror("closing input.txt");
        failed = 1;
    }
    if (failed) {
        free(*matrix);
        *matrix = NULL;
    }
    return failed ? -1 : 0;
}
```

Листинг А.3 – Файл matrix_io.h

```
#ifndef MATRIX_IO_H
#define MATRIX_IO_H
#include <stddef.h>
int read_matrix(size_t *rows, size_t *cols, double **matrix);
#endif
```

Листинг А.4 – Файл benchmark.c

```
#include "benchmark.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
static double seconds_between(struct timespec start, struct timespec end)
{
    return (double)(end.tv_sec - start.tv_sec) +
           (double)(end.tv_nsec - start.tv_nsec) / 1000000000.0;
}
int run_timed_pass(Worker *workers, size_t worker_count, double *total_seconds)
{
    struct timespec start, end;
    if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
        perror("clock_gettime start");
        return -1;
    }
    if (filter_pass(workers, worker_count) != 0)
        return -1;
    if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
        perror("clock_gettime end");
        return -1;
    }
    *total_seconds += seconds_between(start, end);
    return 0;
}
int measure_one_worker(const double *input, double *output, size_t rows,
                       size_t cols, size_t window, size_t iterations,
                       double *seconds)
{
    size_t count = rows * cols;
    double *copy = malloc(count * sizeof(*copy));
    if (copy == NULL) {
        perror("malloc baseline copy");
        return -1;
    }
    memcpy(copy, input, count * sizeof(*copy));
    double *source = copy;
    double *target = output;
    *seconds = 0.0;
    int failed = 0;
    for (size_t pass = 0; pass < iterations; ++pass) {
        Worker worker = {
            .input = source, .output = target, .rows = rows, .cols = cols,
            .window = window, .first_row = 0, .last_row = rows
        };
        if (run_timed_pass(&worker, 1, seconds) != 0) {
            failed = 1;
            break;
        }
        double *temporary = source;
        source = target;
        target = temporary;
    }
```

```
    free(copy);
    return failed ? -1 : 0;
}
```

Листинг А.5 – Файл benchmark.h

```
#ifndef BENCHMARK_H
#define BENCHMARK_H
#include "filter.h"
int run_timed_pass(Worker *workers, size_t worker_count, double *total_seconds);
int measure_one_worker(const double *input, double *output, size_t rows,
                       size_t cols, size_t window, size_t iterations,
                       double *seconds);
#endif
```

Листинг А.6 – Файл filter.c

```
#define _GNU_SOURCE
#include "filter.h"
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
static void *worker_main(void *argument)
// function that applies a matrix filter to a part of the input matrix (in arguments we set rows tha
t will be processed by this worker) and puts the result in the output matrix. The function was made
to be run in a separate thread for each worker
{
    Worker *work = argument;
    size_t radius = work->window / 2;
    double weight = 1.0 / (double)(work->window * work->window);
    work->tid = gettid();
    for (size_t row = work->first_row; row < work->last_row; ++row) {
        for (size_t col = 0; col < work->cols; ++col) {
            double sum = 0.0;
            for (size_t kr = 0; kr < work->window; ++kr) {
                if (row + kr < radius || row + kr - radius >= work->rows)
                    continue;
                size_t source_row = row + kr - radius;
                for (size_t kc = 0; kc < work->window; ++kc) {
                    if (col + kc < radius || col + kc - radius >= work->cols)
                        continue;
                    size_t source_col = col + kc - radius;
                    sum += work->input[source_row * work->cols + source_col] * weight;
                }
            }
            work->output[row * work->cols + col] = sum;
        }
    }
    return NULL;
}
int filter_pass(Worker *workers, size_t count)
{
    pthread_t *threads = malloc(count * sizeof(*threads));
    if (threads == NULL) {
        perror("malloc threads");
        return -1;
    }
    size_t created = 0;
    int failed = 0;
    for (; created < count; ++created) {
        int error = pthread_create(&threads[created], NULL, worker_main, &workers[created]);
        if (error != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(error));
            failed = 1;
            break;
        }
    }
    for (size_t i = 0; i < created; ++i) {
        int error = pthread_join(threads[i], NULL);
        if (error != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(error));
            failed = 1;
        }
    }
    free(threads);
```

```
    return failed ? -1 : 0;
}
```

Листинг А.7 – Файл filter.h

```
#ifndef FILTER_H
#define FILTER_H
#include <stddef.h>
#include <sys/types.h>
typedef struct {
    const double *input;
    double *output;
    size_t rows;
    size_t cols;
    size_t window;
    size_t first_row;
    size_t last_row;
    pid_t tid;
} Worker;
int filter_pass(Worker *workers, size_t count);
#endif
```

Листинг А.8 – Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(lab3 LANGUAGES C)
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
find_package(Threads REQUIRED)
add_executable(lab3
    main.c
    filter.c
    matrix_io.c
    benchmark.c
)
target_compile_definitions(lab3 PRIVATE _POSIX_C_SOURCE=200809L)
target_compile_options(lab3 PRIVATE -Wall -Wextra -Wpedantic)
target_link_libraries(lab3 PRIVATE Threads::Threads m)
```

Приложение Б. Полный вывод strace

Полный вывод выбранной трассировки strace без пропущенных строк. Номера PID/TID и адреса относятся к одному конкретному запуску и при следующем запуске будут другими.

Листинг Б.1 – Вывод strace для лабораторной работы 3

```
42385 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
42385 close(3)                          = 0
42385 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
42385 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
42385 close(3)                          = 0
42385 openat(AT_FDCWD, "input.txt", O_RDONLY) = 3
42385 read(3, "3 3\n1 1 1\n1 1 1\n1 1 1\n", 4096) = 22
42385 close(3)                          = 0
42385 read(0, "3\r\n", 4096)            = 3
42385 read(0, "1\r\n", 4096)            = 3
42385 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SET
TLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x777b793ff990, parent_tid=0x777b793ff990, e
xit_signal=0, stack=0x777b78bff000, stack_size=0x7fff80, tls=0x777b793ff6c0} => {parent_tid=[42390]}
, 88) = 42390
42385 futex(0x777b793ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 42390, NULL, FUTEX_BITSET_MATCH_
ANY <unfinished ...>
42390 gettid()                          = 42390
42385 <... futex resumed>)              = 0
42390 +++ exited with 0 +++
42385 openat(AT_FDCWD, "output.txt", O_WRONLY|O_CREAT|O_APPEND, 0666) = 3
42385 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SET
TLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x777b793ff990, parent_tid=0x777b793ff990, e
xit_signal=0, stack=0x777b78bff000, stack_size=0x7fff80, tls=0x777b793ff6c0} => {parent_tid=[42391]}
, 88) = 42391
42391 gettid()                          = 42391
42385 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SET
TLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x777b78bfe990, parent_tid=0x777b78bfe990, e
xit_signal=0, stack=0x777b783fe000, stack_size=0x7fff80, tls=0x777b78bfe6c0} => {parent_tid=[42392]}
, 88) = 42392
42385 futex(0x777b793ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 42391, NULL, FUTEX_BITSET_MATCH_
ANY <unfinished ...>
42391 +++ exited with 0 +++
42385 <... futex resumed>)              = -1 EAGAIN (Resource temporarily unavailable)
42385 futex(0x777b78bfe990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 42392, NULL, FUTEX_BITSET_MATCH_
ANY <unfinished ...>
42392 gettid()                          = 42392
42385 <... futex resumed>)              = 0
42392 +++ exited with 0 +++
42385 write(3, "\nMatrix 3 x 3, window = 3, K = 1"..., 212) = 212
42385 close(3)                          = 0
42385 write(1, "Enter window size:\nEnter number "..., 179) = 179
42385 +++ exited with 0 +++
```